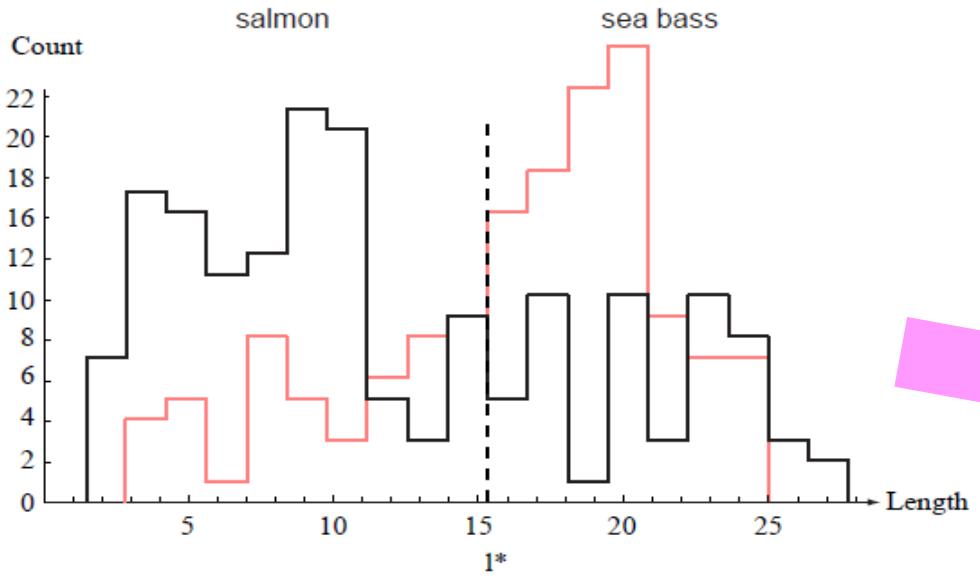
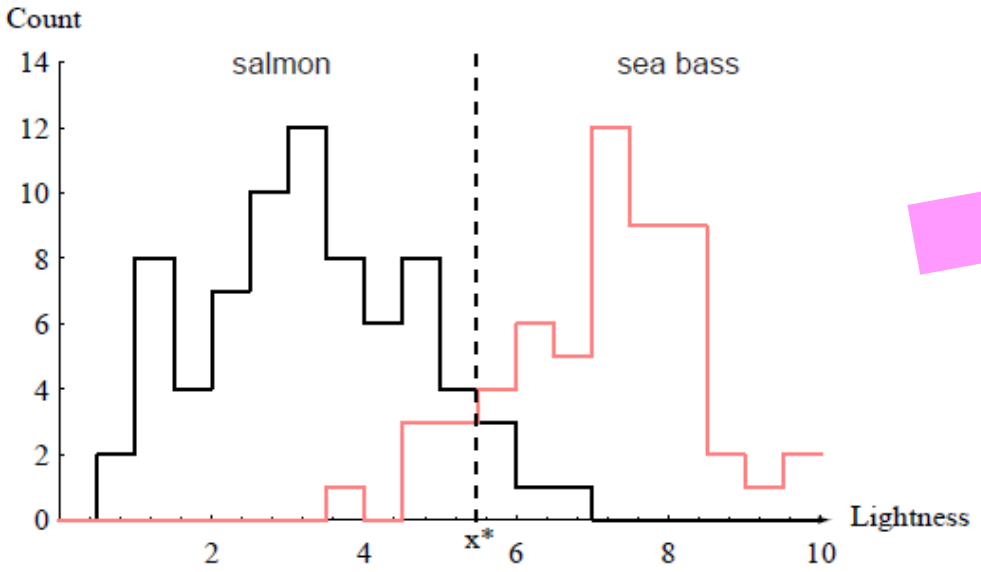
Feature Space Geometry

Distances in the Feature Space

- The first "concept" we need for the feature space is "distance":
  - How do we measure the distance between two samples?
- Let us consider the role of distance in classification using the simplest classifier: 1NN (nearest-neighbor) classifier:
  - A query sample is assigned to the class of its nearest (minimum distance) neighbor in the training set.
- Question: How do we define or compute the distance?
  - Euclidean distance is the most common one when the features are continuous.

The Sample Two-Class Problem

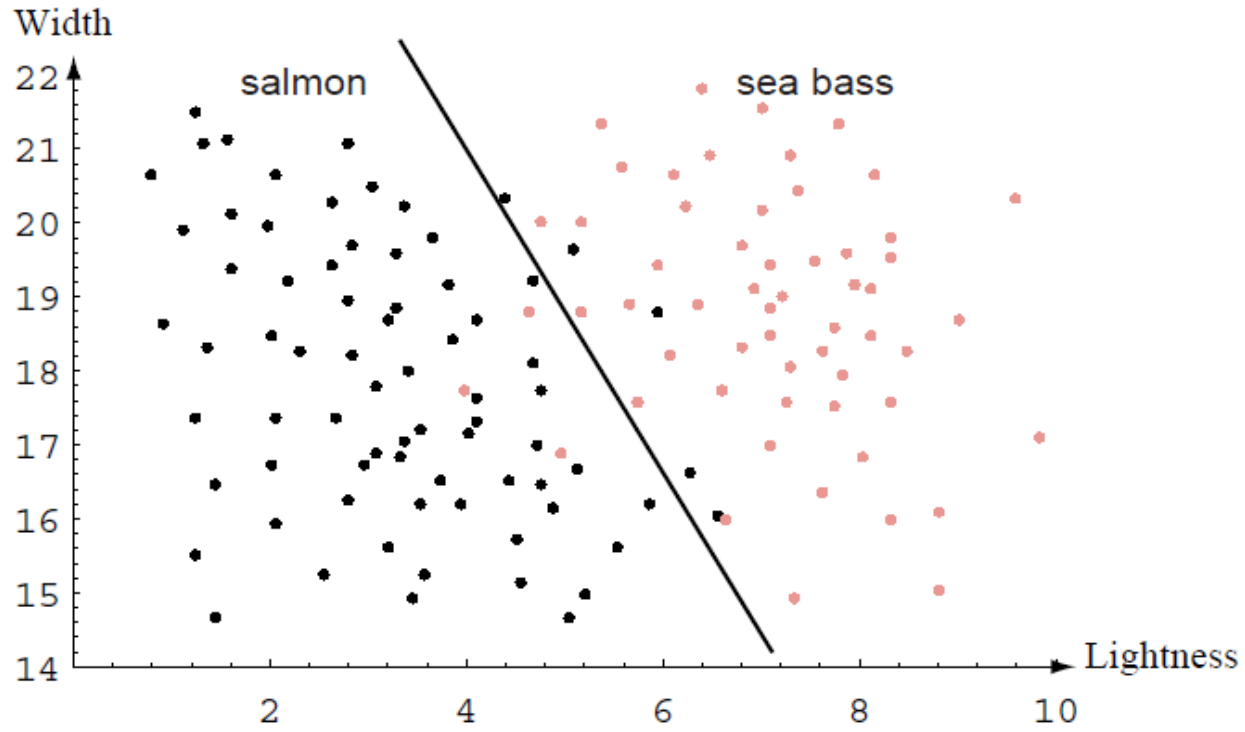




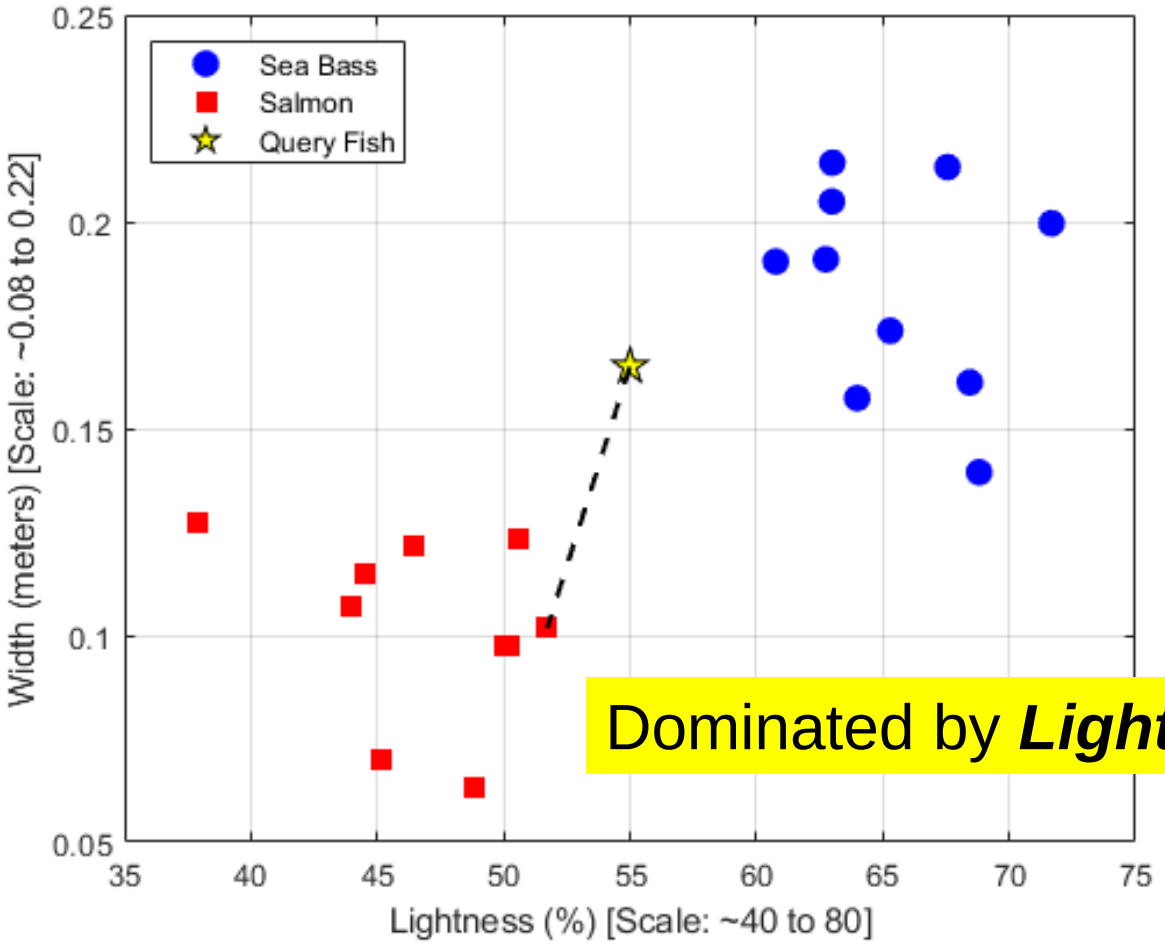




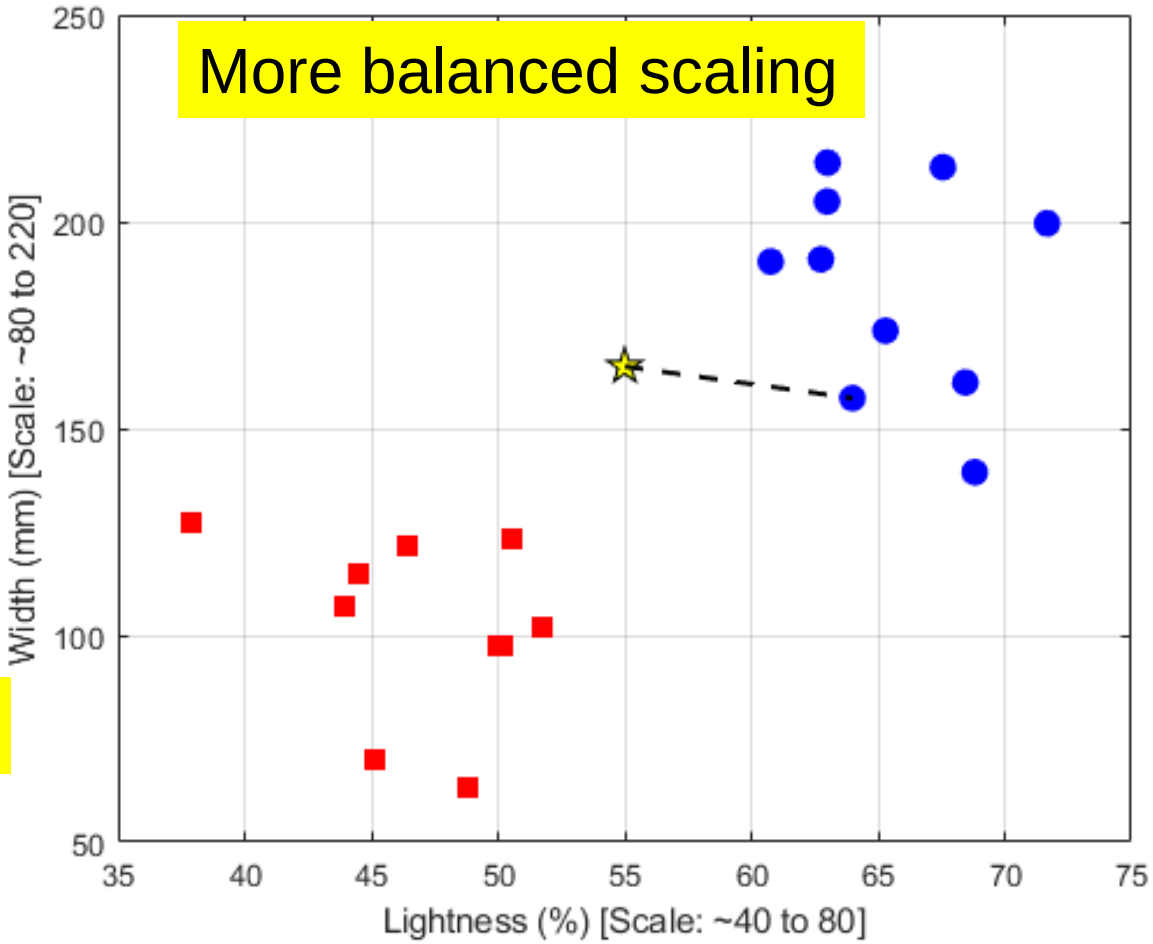
Feature Space



Nearest-Neighbor Classification



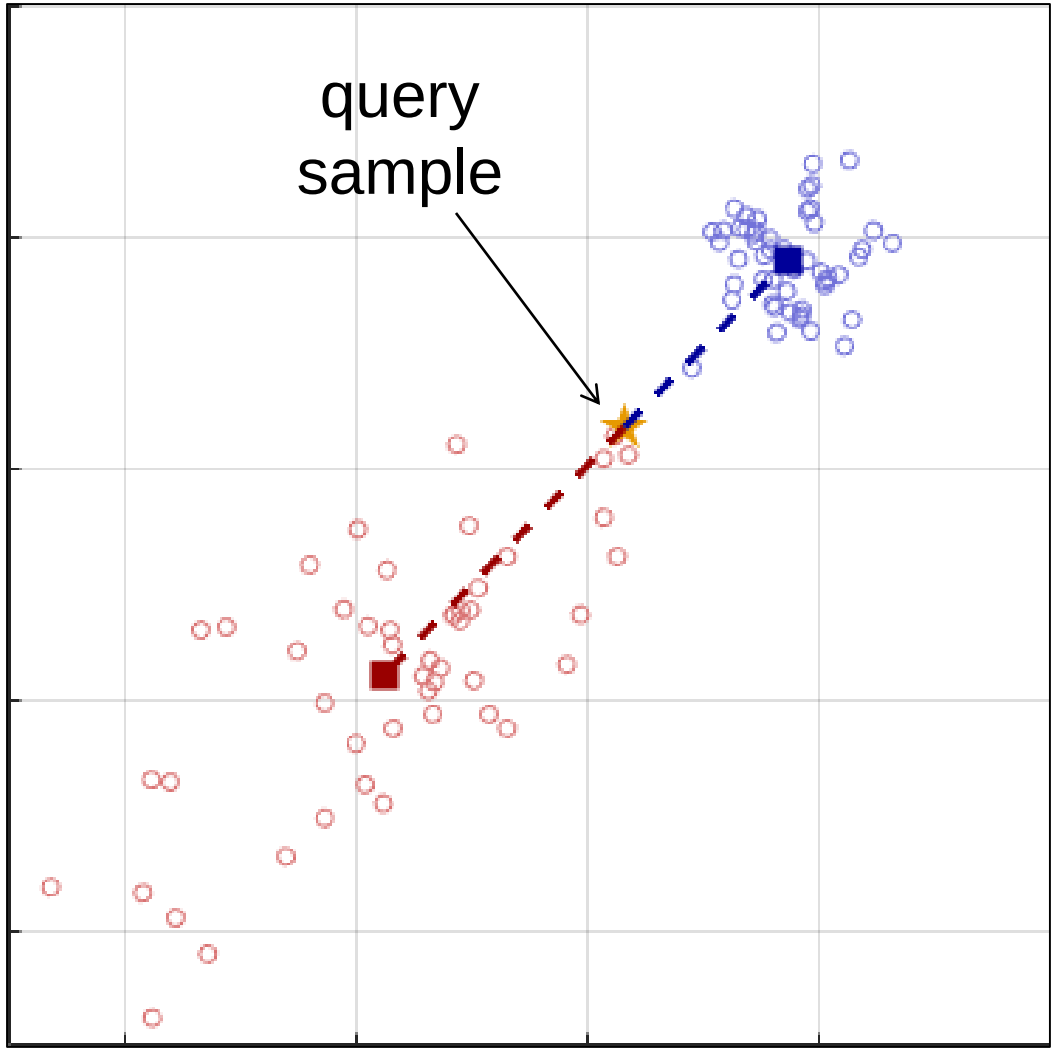
Dominated by ***Lightness***



This problem is commonly handled with **data normalization**:

Scale and shift the features so they all have similar means and variances.

- However, data normalization is insufficient.
- The problem of classifying the query sample here:
  - Using Euclidean distances to the "nearest classes" (e.g., class centroids here) might not be reasonable.
- Question: How to handle variations in data distribution when computing ***distances***?



Inner Products

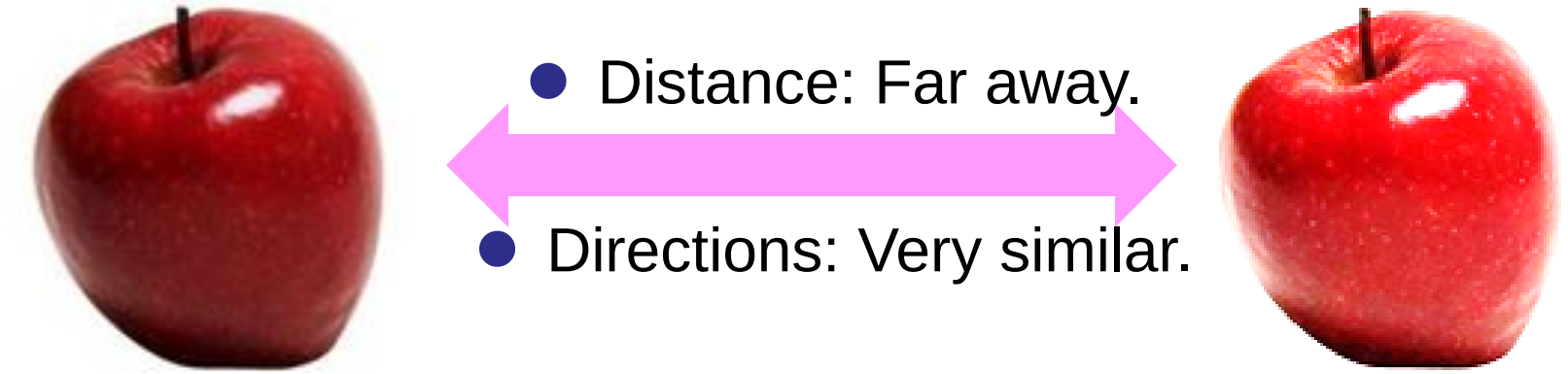
- We start with inner products because they are the key of defining the properties of a space.
- This is the standard inner product definition:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \sum x_i y_i$$

- Instead of just computing inner products of vectors, we use inner products to define ***distances*** and ***angles***:
  - Distance / norm: $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$
  - Angle: $\cos(\theta) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$
  - Orthogonality: When the inner product is zero.

Inner Products

- Directional Alignment (angle) is more commonly used for comparing feature vectors than distances.
- Example: Lighting Trap:



Metric-Tensor Inner Products

- The standard (Euclidean) inner products work when:
  - All the dimensions are equally important;
  - All the dimensions are uncorrelated (orthogonal).
- When these do not apply, we need a "warped" inner product defined with a metric-tensor matrix $\mathbf{M}$: $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{M}} = \mathbf{x}^T \mathbf{M} \mathbf{y}$
- Required properties of $\mathbf{M}$:
  - Symmetry.
  - Positive definiteness ($\mathbf{x}^T \mathbf{M} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$)

Redefining Orthogonality: Example (1)

- Example: A signal mixing problem:
  - Physical states (uncoupled): s and t;
  - Sensor readings (coupled): $u = s$ and $v = s+t$
- Assume the following "apparently orthogonal" sensor readings:

$$\mathbf{a}_{\text{sensor}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ and } \mathbf{b}_{\text{sensor}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- They correspond to physical states $\mathbf{a}_{\text{phys}} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \mathbf{b}_{\text{phys}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$
  - Not orthogonal! (They are actually correlated!)

Redefining Orthogonality: Example (2)

- Assume another set of "apparently non-orthogonal" sensor readings:

$$\mathbf{a}_{\text{sensor}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{ and } \mathbf{b}_{\text{sensor}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- They correspond to physical states

$$\mathbf{a}_{\text{phys}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{b}_{\text{phys}} = A^{-1} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

  - Actually orthogonal!

- We use this metric-tensor matrix (derivation skipped):

$$\begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix}$$

Redefining Orthogonality: Example (3)

- Now for the first set of readings (apparently orthogonal):

$$\langle \mathbf{a}_{\text{sensor}}, \mathbf{b}_{\text{sensor}} \rangle_M = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -1$$

- Now for the second set of readings (apparently non-orthogonal):

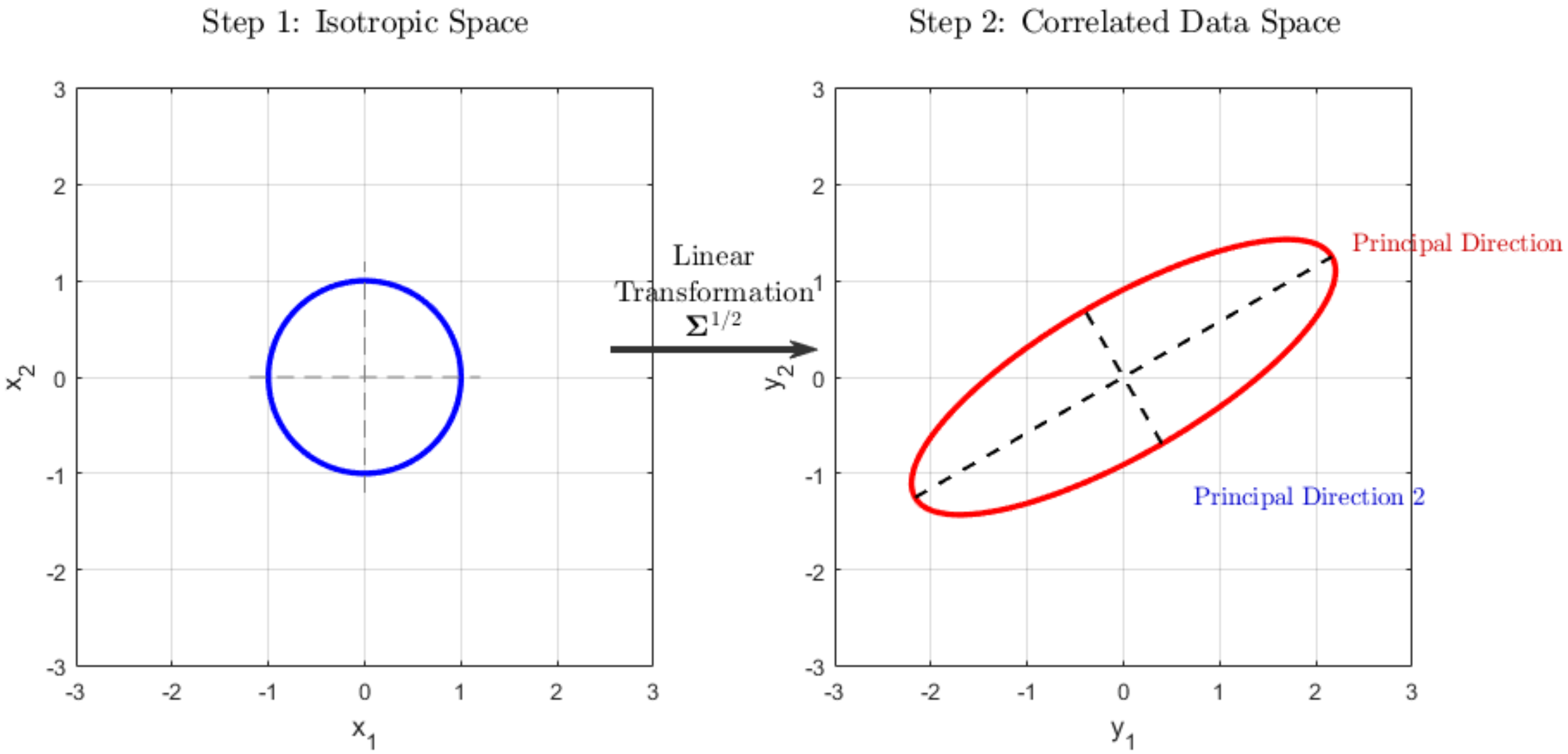
$$\langle \mathbf{a}_{\text{sensor}}, \mathbf{b}_{\text{sensor}} \rangle_M = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = 0$$

Covariance Matrix for Spatial Transform

$$\boldsymbol{\Sigma} = \frac{1}{N}\mathbf{X}^T\mathbf{X}$$

- Classical View (Statistics): Covariance is a passive summary table recording variances along diagonal entries and correlations on off-diagonal elements.
- Our View (Geometric PR): Covariance is an active geometric transformation matrix. It describes how a raw, isotropic unit sphere of random noise was stretched, sheared, and rotated to generate the observed data cloud.

Covariance Matrix for Spatial Transform



Covariance Matrix for Spatial Transform

- Meaning of spectral decomposition:

$$\Sigma = \mathbf{V}\Lambda\mathbf{V}^T$$

rotate to alignment

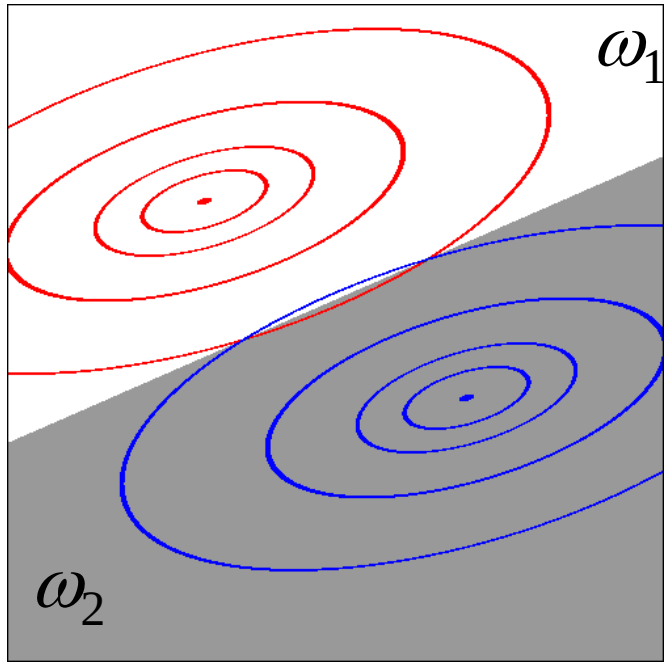
axes scaling

rotate back

- Rotation: To align with the principle axes (eigenvectors)
- Scaling: (diagonal matrix) with eigenvalues

Mahalanobis Distances

- We used to understand Mahalanobis distance as a distance to a normal distribution:



d_m shown:
(from inside out)
- –ln(0.8)
- –ln(0.5)
- –ln(0.1)
- –ln(0.001)

- We now consider it as the distance in a **new inner product space** with the metric tensor matrix $\mathbf{M} = \mathbf{\Sigma}^{-1}$:

$$\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{\Sigma}^{-1}} = \mathbf{x}^T \mathbf{\Sigma}^{-1} \mathbf{y} \quad \Rightarrow \quad D_M(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \mathbf{\Sigma}^{-1} (\mathbf{x} - \mathbf{y})}$$

Mahalanobis Distances

- Spatial Transform from the covariance matrix:

$$\mathbf{\Sigma} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$$

$$\mathbf{\Sigma}^{-1} = \mathbf{V}\mathbf{\Lambda}^{-1/2}\mathbf{\Lambda}^{-1/2}\mathbf{V}^T = \mathbf{A}^T\mathbf{A} \quad \text{where} \quad \mathbf{A} = \mathbf{\Lambda}^{-1/2}\mathbf{V}^T$$

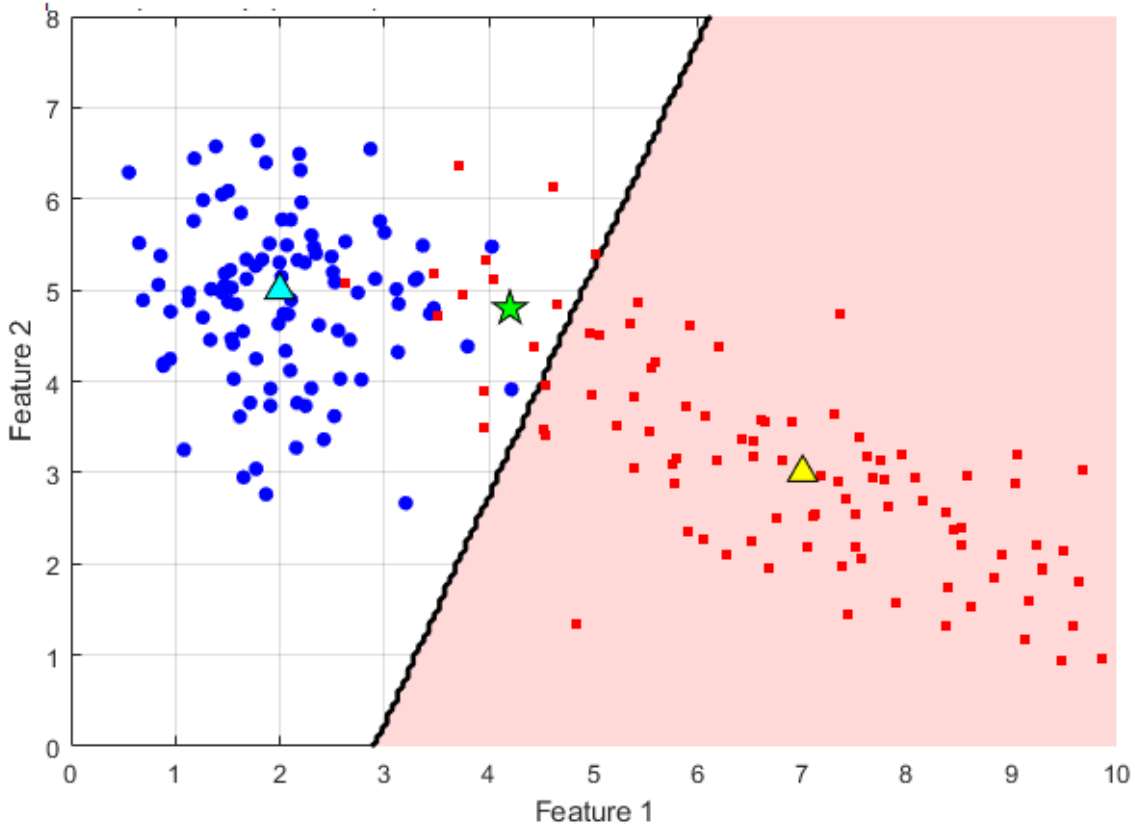
- Transformed points:

$$\mathbf{z} = \mathbf{A}\mathbf{x} = \mathbf{\Lambda}^{-1/2}\mathbf{V}^T\mathbf{x}$$

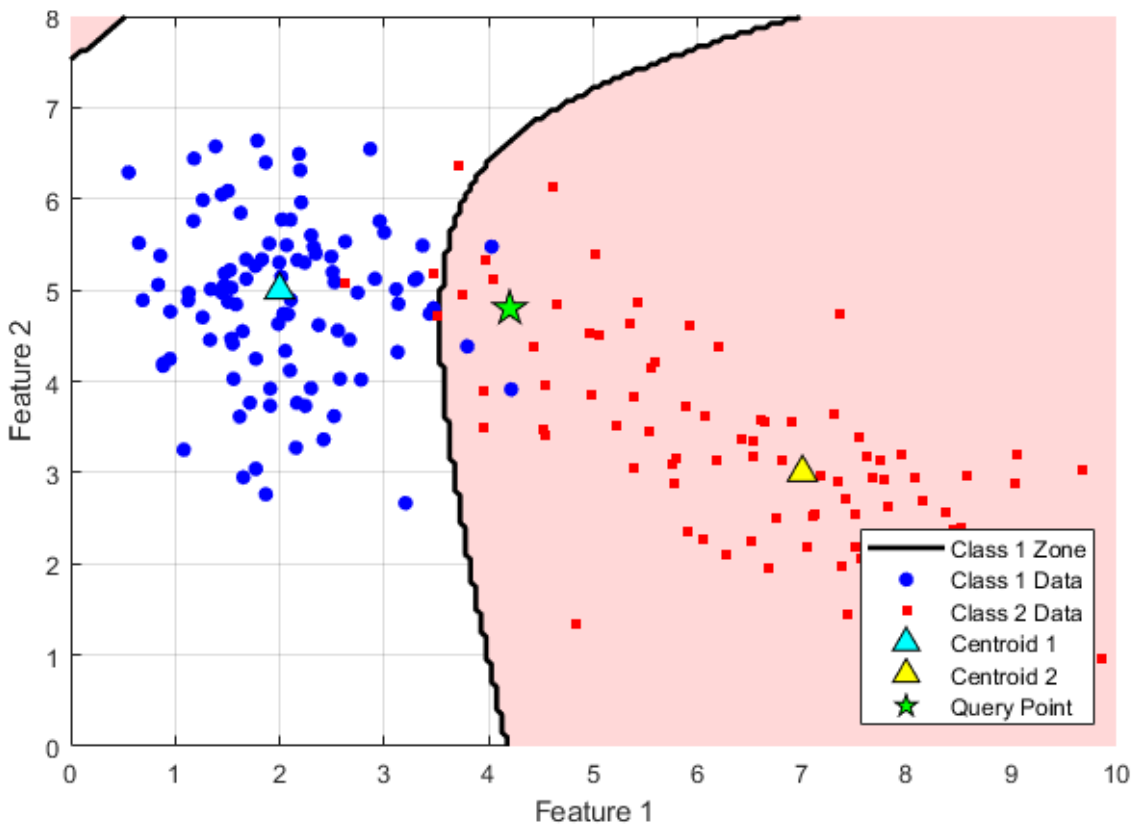
- The Mahalanobis distance between original points is the Euclidean distance between transformed points.

Mahalanobis Distances and Classification

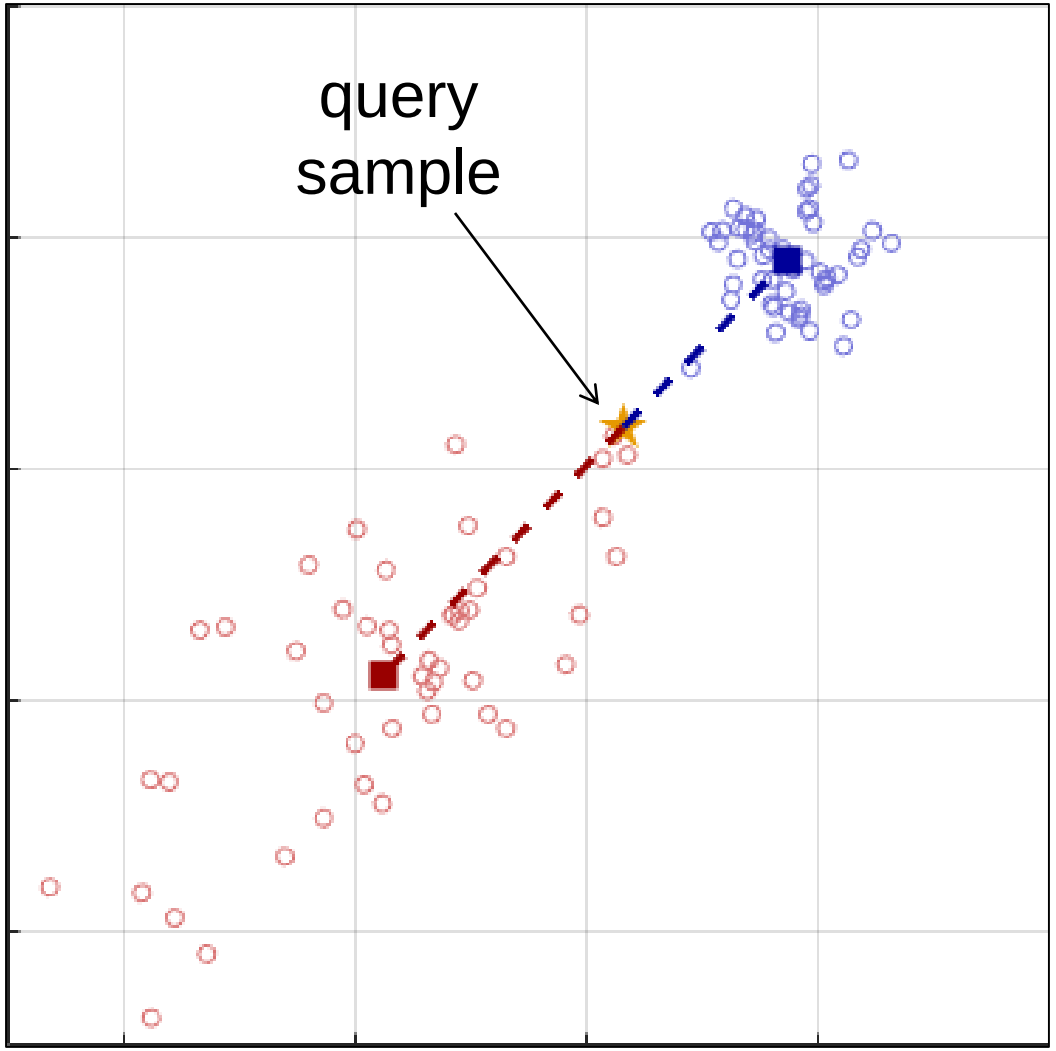
Nearest-centroid, Euclidean distance

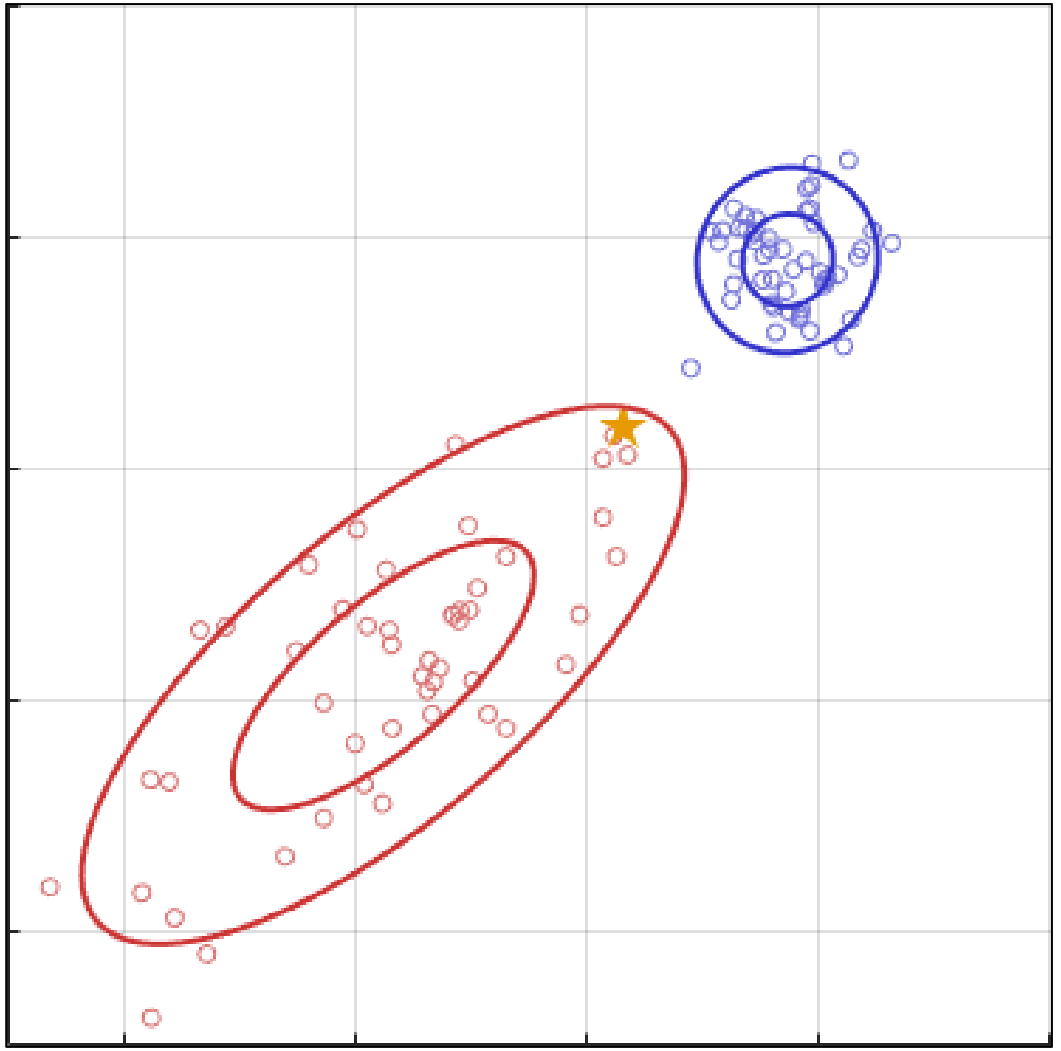


Nearest-centroid, Mahalanobis distance



Mahalanobis Distances and Classification





Mahalanobis Distances and Bayes Classifiers

- The Bayes Rule: $P(\omega_i \mid \mathbf{x}) \propto p(\mathbf{x} \mid \omega_i) P(\omega_i)$
- Discriminant function based on the Bayes Rule: $g_i(\mathbf{x}) = \ln p(\mathbf{x} \mid \omega_i) + \ln P(\omega_i)$
- Assume a Gaussian distribution:

$$p(\mathbf{x} \mid \omega_i) = \frac{1}{(2\pi)^{d/2} |\mathbf{\Sigma}_i|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{\Sigma}_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i)\right)$$

- Discriminant function:

$$g_i(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_i)^T \mathbf{\Sigma}_i^{-1} (\mathbf{x} - \boldsymbol{\mu}_i) - \frac{1}{2} \ln |\mathbf{\Sigma}_i| + \ln P(\omega_i) - \frac{d}{2} \ln(2\pi)$$

Squared Mahalanobis distance

- Equal variances & equal Priors:

Bayes Decision Rule ⟺ Minimizing Mahalanobis Distance

- Otherwise: Minimizing Mahalanobis Distance with penalty terms

A Pointer to Metric Learning

- For Mahalanobis distance / norm, we obtain the metric tensor matrix from the covariance matrix.
- In general, we can also obtain the matrix from
  - Known distortion matrix (such as known sensor mixing):

$$\mathbf{M} = \mathbf{A}^T \mathbf{A}$$

  - Supervised optimization (**metric learning**)

Spatial Whitening

- Definition: A linear transformation that converts a random vector with covariance into a **zero-mean vector** whose covariance matrix is the **Identity Matrix**.
- Geometric Meaning: Whitening eliminates all feature correlations (off-diagonals 0) and standardizes all feature variances to 1 (diagonals 1). It transforms an anisotropic, elongated data ellipse into a perfectly round, isotropic unit sphere

Spatial Whitening: PCA

- PCA (Principle Component Analysis) is the transform that connects Mahalanobis distance (original space) to Euclidean distance (whitened space):

$$\boldsymbol{\Sigma} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^T$$

$$\mathbf{W}_{\text{PCA}} = \boldsymbol{\Lambda}^{-1/2}\mathbf{V}^T \implies \mathbf{z}_{\text{PCA}} = \boldsymbol{\Lambda}^{-1/2}\mathbf{V}^T\mathbf{x}$$

$$\|\mathbf{z}_{\text{PCA}}\|_2^2 = \mathbf{x}^T\left(\mathbf{V}\boldsymbol{\Lambda}^{-1/2}\right)\left(\boldsymbol{\Lambda}^{-1/2}\mathbf{V}^T\right)\mathbf{x} = \mathbf{x}^T\boldsymbol{\Sigma}^{-1}\mathbf{x} = D_M(\mathbf{x},\mathbf{0})^2$$

- We'll get into dimensionality reduction later.

Spatial Whitening: ZCA

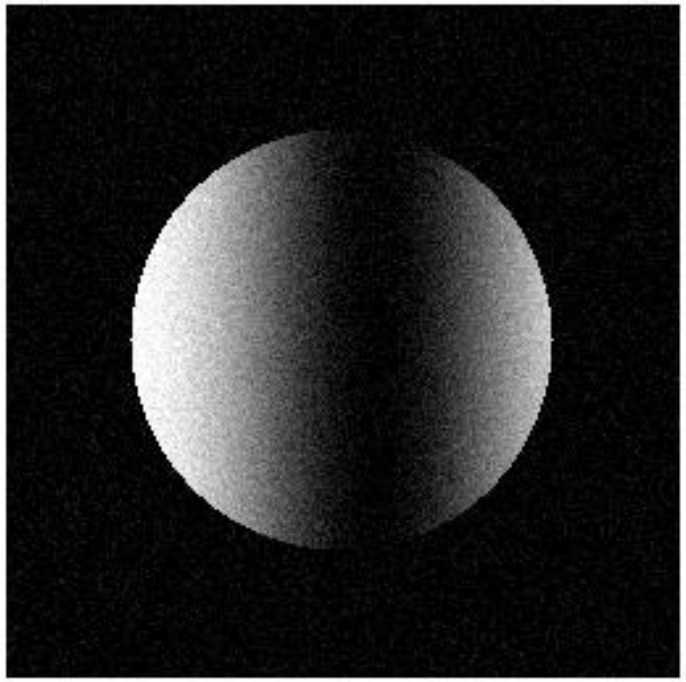
- ZCA (Zero-Phase Component Analysis) applies an additional rotation back to the PCA-whitened space:

$$\mathbf{W}_{\text{ZCA}} = \mathbf{V}\mathbf{W}_{\text{PCA}} = \mathbf{V}\mathbf{\Lambda}^{-1/2}\mathbf{V}^T = \mathbf{\Sigma}^{-1/2}$$

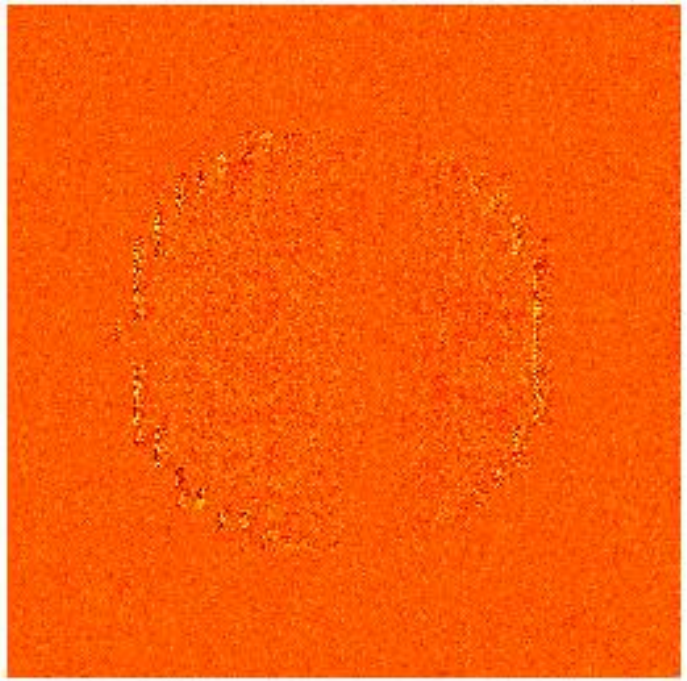
- Rationales:
  - PCA whitens space but rotates the axes into abstract principal directions, making the features uninterpretable in the original domain.
  - ZCA minimizes the squared reconstruction error between original and whitened vectors.
  - Result: ZCA yields whitened data that looks as close as possible to the original inputs.

Spatial Whitening Example: PCA vs. ZCA

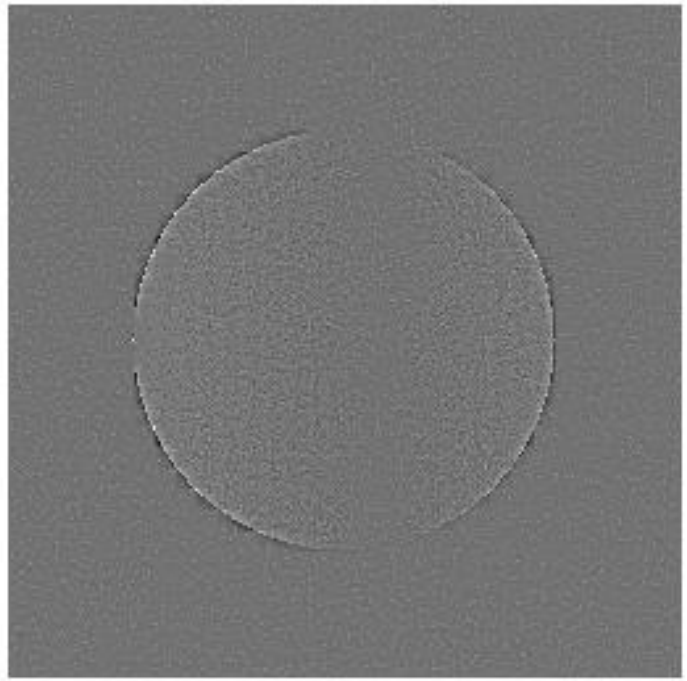
Original Image



PCA of Patches



ZCA of Patches



Example: 256x256 image with 8x8 patches as feature vectors

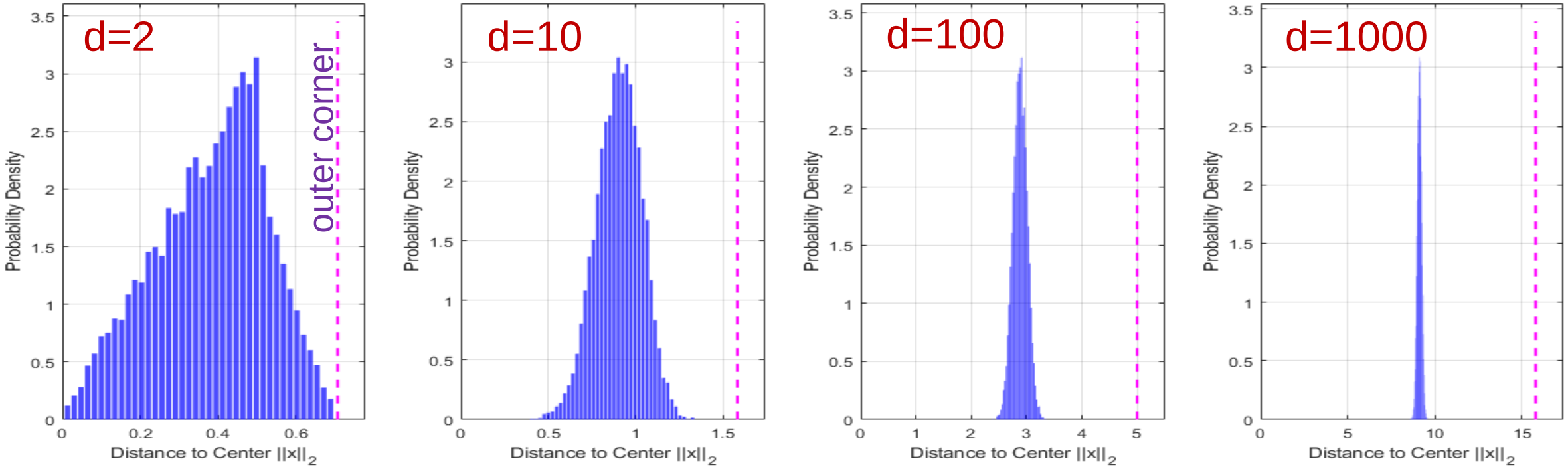
High-Dimensional Space

- **Curse of Dimensionality**: It's not just about computational expense or overfitting.
- Properties of a high-dimensional space:
  - Data Sparsity ➔ Distances and angles lose meaning (all sample pairs have similar distances and vector pairs are all close to orthogonal).
  - Many dimensions have low eigenvalues ➔ Rank-deficient covariance ➔ Whitening fails because of exploding noise directions (scaled by inverse of small eigenvalues).
- Conclusion: **Dimensionality Reduction** is necessary!

High-Dimensional Space

An illustration of why "distance" becomes meaningless:

The histogram of distance-to-center of random points within a unit hypercube.



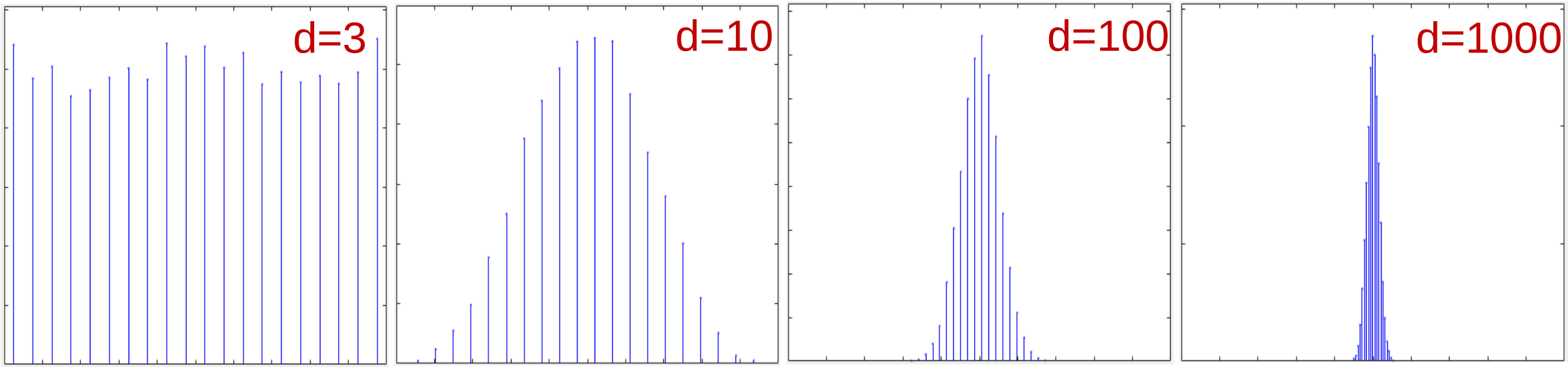
It appears that all the points are on a very thin shell in a high-dimensional space.

The "interior" is what is called "high-dimensional void".

High-Dimensional Space

An illustration of why "angle" becomes meaningless:

The histogram of cosine similarity of random point pairs on a unit hypersphere.

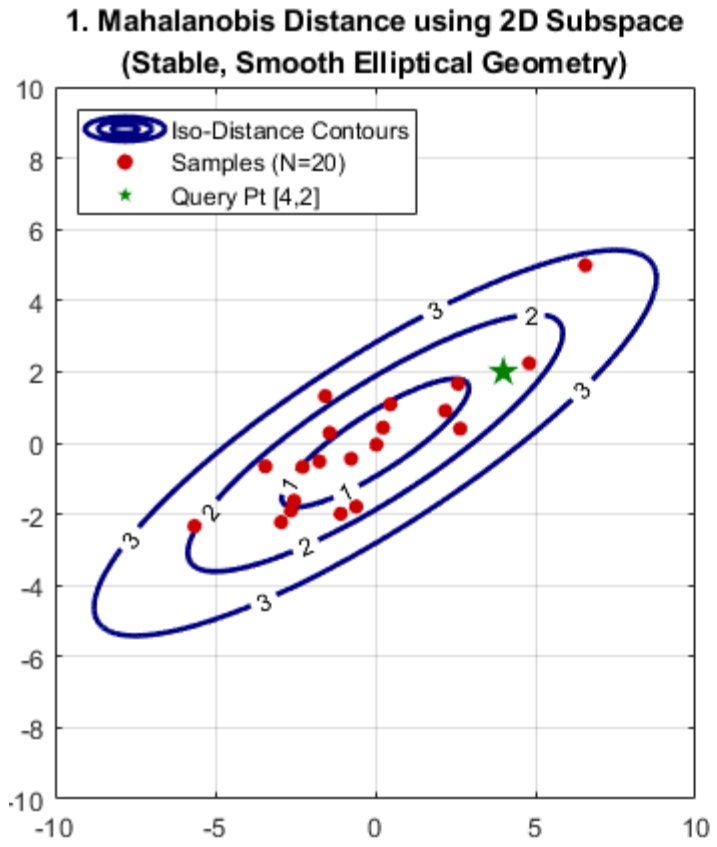


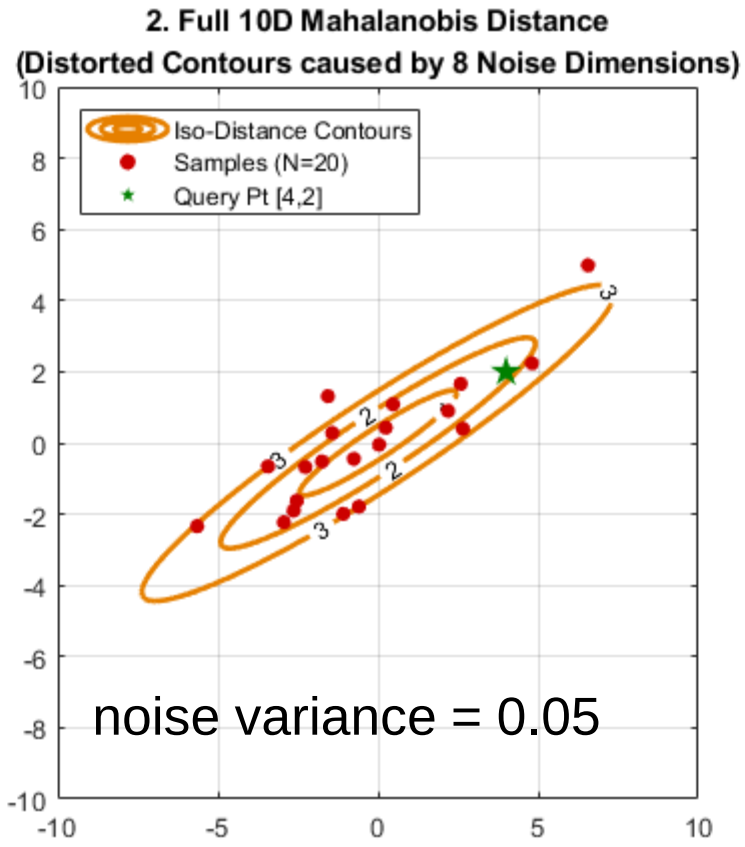
(horizontal axis: -1 ~ 1)

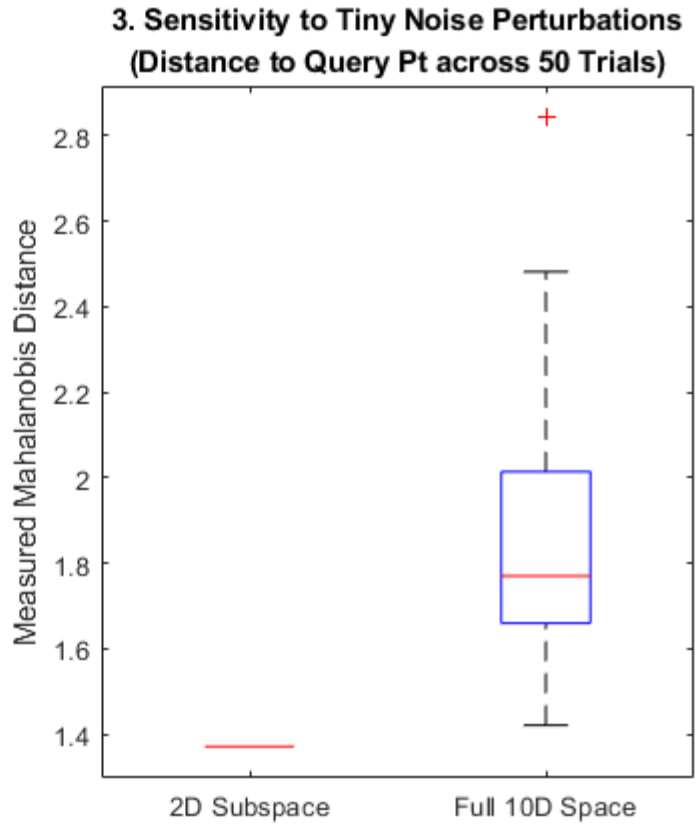
It appears that all point pairs are orthogonal. Every task based on "similarity" becomes unreliable.

Mahalanobis Distances with Noise Dimensions

An illustration: 20 random 10-D points (two dominant and 8 noise dimensions)







Truncated PCA for Dimensionality Reduction

We follow the derivation based on minimizing the **reconstruction error**:

The cost function of PCA, which is the total squared errors of reconstruction (projection of the transformed points into the original space):

$$J = \sum_{i=1}^{N} \left\| \boldsymbol{x}_i - \sum_{j=1}^{l} (\boldsymbol{x}_i^T \boldsymbol{e}_j) \boldsymbol{e}_j \right\|^2$$

with the constraints

$$\boldsymbol{e}_i{}^T \boldsymbol{e}_j = 0 \quad \text{for } \forall i \neq j$$

$$\boldsymbol{e}_i{}^T \boldsymbol{e}_i = 1 \quad \text{for } \forall i$$

Here $\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_l$ are the l orthonormal basis vectors of the subspace.

The projection of $\boldsymbol{x}$ onto this subspace is given by $\sum_{j=1}^{l} (\boldsymbol{x}_i^T \boldsymbol{e}_j) \boldsymbol{e}_j$

Truncated PCA for Dimensionality Reduction

The Lagragian after incorporating the constraints and some manipulation:

$$L = \sum_{i=1}^{N} \| \boldsymbol{x}_i \|^2 - \sum_{j=1}^{l} \left(\boldsymbol{e}_j{}^T \Sigma_x \boldsymbol{e}_j \right) - \sum_{j=1}^{l} \lambda_j \left(\boldsymbol{e}_j{}^T \boldsymbol{e}_j - 1 \right)$$

Setting $\frac{\partial L}{\partial \boldsymbol{e}_j} = \boldsymbol{0}$ results in the eigenvalue problem: $\Sigma_x \boldsymbol{e}_j = \lambda_j \boldsymbol{e}_j$

Putting this back into the original cost function, we get $J = \sum_{i=1}^{N} \| \boldsymbol{x}_i \|^2 - \sum_{j=1}^{l} \lambda_j$

Implication: To minimize the re-projection error, we choose the eigenvectors that correspond to the l largest eigenvalues of Σ_x to define the projection.

Example PCA Application: Eigenfaces

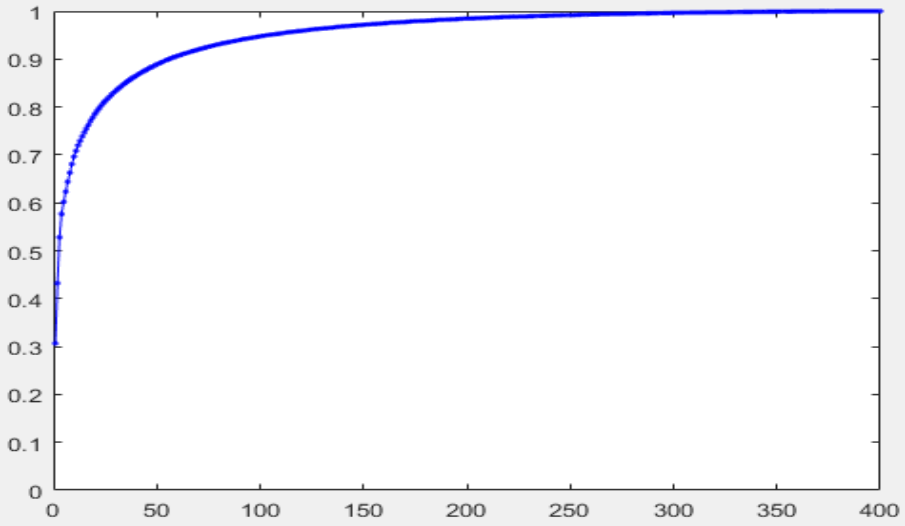
An example application of PCA is in face recognition, where each input vector has the dimension of the number of pixels in a face image. A popular technique is "eigenface", where the most important eigenvectors obtained by PCA are used as the basis vectors. The recognition is then based on the projection of an input image to these "eigenfaces".



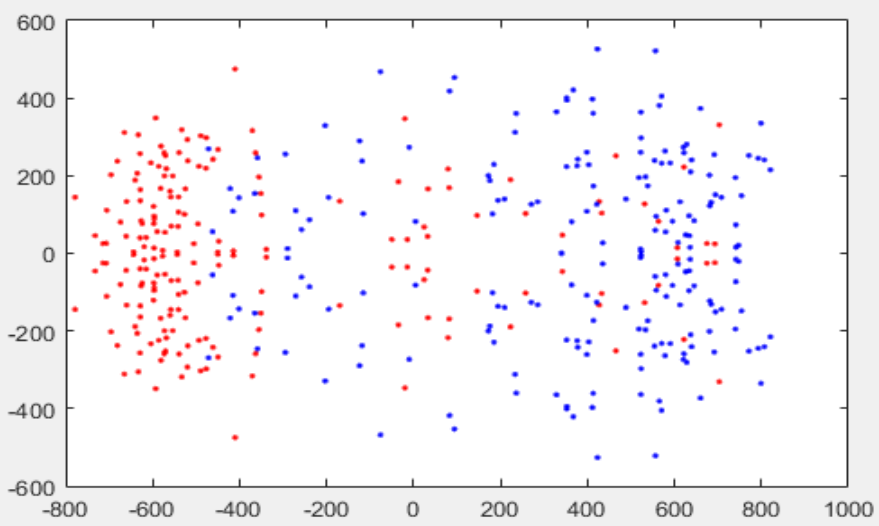
200 images
(100 males + 100 females)
+ horizontally flipped images
size 40x40
➔ 400 1600-D vectors

Example PCA Application: Eigenfaces

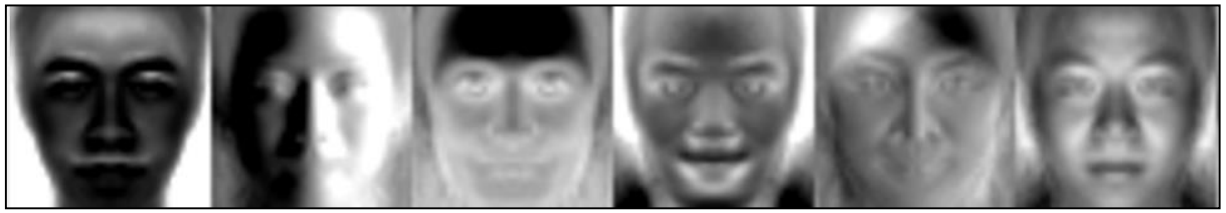
Accumulated variances (normalized):



Projections in the first two PCA dimensions:



The first few eigenfaces:



Reconstructed faces along the first PCA dimension:

